TRACHCARE

A PRODUCT DESIGN AND DEVELOPMENT REPORT

Submitted to

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

In partial fulfillment of the award of the degree of

BACHELOR OF ENGINEERING

By

Name and Reg Number: _____ - _____

Name and Reg Number: _____ - _____

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI - 602 105

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI - 602105

BONAFIDE CERTIFICATE

This product development report "TRACHCARE" is the Bonafide work of _____ who carried out the product development work under my supervision.

Program Director
Information Technology
SIMATS Engineering
Saveetha Institute of Medical
and Technical Sciences
Chennai - 602105

SUPERVISOR
Information Technology
SIMATS Engineering
Saveetha Institute of Medical
and Technical Sciences
Chennai - 602105

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION BY THE CANDIDATE

I declare that the project entitled "TRACHCARE" submitted by me for the SPIC7 - Product Design and Development course is our original work. We carried out this project under the guidance of our supervisor and it has been completed successfully.

Signature of the Candidate

_____ _____

_____ _____

INDEX

S.NO	TOPICS	PAGE NO
1	Abstract	i
2	List of Figures	iii
3	List of Tables	iv
4	Chapter 1 - Introduction	1
5	Chapter 2 - Gap Analysis, Product Description, Comparison and Uniqueness	5
6	Chapter 3 - Requirements	10
7	Chapter 4 - Technology Stack	13
8	Chapter 5 - User Experience Design	16
9	Chapter 6 - System Architecture and Design	20
10	Chapter 7 - Implementation	24
11	Chapter 8 - Project Management and Teamwork	33
12	Chapter 9 - Testing and Validation	36
13	Chapter 10 - Deployment	39
14	Chapter 11 - Security, Privacy and Clinical Responsibility	43
15	Chapter 12 - Limitations, Future Work and Conclusion	46
16	References	48

Abstract

TrachCare is an Android application that connects tracheostomy patients recovering at home with the doctors who monitor them and the administrators who run the service. Patients submit a structured daily health report - respiratory rate, heart rate, SpO2, tube and surgery status, comorbidities and lab values - from their phone, while doctors review every submission, manage their patient list and receive flagged reports for abnormal entries.

After a tracheostomy, patients are discharged with instructions and a paper diary. Care steps are easy to forget, readings are written inconsistently, and the treating doctor sees the patient only at scheduled reviews - sometimes weeks apart. Deterioration between visits is invisible until it becomes an emergency.

TrachCare replaces the paper diary with a guided workflow:

- The patient logs daily vitals and care status through a structured form.
- The record reaches the treating doctor, who reviews entries and sees flagged reports first.
- A therapy-video library published by the admin trains patients and caregivers on correct care procedures.
- An administrator approves new doctor accounts, manages users and curates the education content.

The app is built with Flutter (Android), a PHP REST backend and a MariaDB database running on XAMPP, secured with role-based login, OTP verification and an admin approval workflow. It is released as a signed Android App Bundle targeting API level 36 through Google Play Console.

Keywords: tracheostomy, home care, remote patient monitoring, Flutter, PHP, MySQL, XAMPP, role-based access control, daily health report

List of Figures

Figure	Title	Image name	Chapter
5.1	Hand-drawn wireframes (login, report, monitor, admin)	fig_5_2_wireframes.png	5
5.2	User flow across the three roles	fig_flow.png	5
6.1	System architecture of TrachCare	fig_6_1_architecture.png	6
6.2	Database tables and key fields	fig_6_2_schema.png	6
7.1	Splash screen with ECG trace	fig_7_1_splash.png	7
7.2	Patient sign-in screen	fig_7_2_login.png	7
7.3	Patient dashboard	fig_7_3_patient_dash.png	7
7.4	Daily update form	fig_7_4_daily_update.png	7
7.5	Doctor dashboard with patient list	fig_7_5_doctor_dash.png	7
7.6	Add patient chart form	fig_7_6_add_patient.png	7
7.7	Admin approvals queue	fig_7_7_approvals.png	7
7.8	Therapy video library	fig_7_8_videos.png	7
8.1	Project timeline	fig_timeline.png	8

List of Tables

Table	Title	Chapter
2.1	Weaknesses of paper-based tracheostomy care tracking	2
2.2	Users and roles	2
2.3	Comparison of TrachCare with existing alternatives	2
3.1	Functional requirements	3
3.2	Non-functional requirements	3
4.1	Technology stack	4
4.2	Hardware and software requirements	4
5.1	Design review findings and changes	5
6.1	Database tables and key fields	6
7.1	PHP REST API endpoints	7
7.2	Module summary	7
8.1	Project stages	8
8.2	Team roles and responsibilities	8
9.1	Test cases and results	9
11.1	Security threats and controls	11
12.1	Future enhancements	12

Chapter 1 - Introduction

1.1 Background

A tracheostomy creates a surgical airway through the front of the neck and is performed for patients who need long-term ventilation support, airway protection or secretion management. After discharge, the patient and a family caregiver take over daily care: keeping the tube patent, changing the dressing, suctioning secretions, watching the stoma for infection and tracking vital signs that the treating team reviews periodically.

TrachCare is a tracheostomy care and monitoring application designed to support exactly that period between hospital and home. The patient (or caregiver) submits a structured daily health report from an Android phone; the treating doctor reviews submissions the same day through a monitoring console; and an administrator manages doctor approvals, user lists and a guided therapy-video library.

The product has three coordinated parts: a Flutter mobile application with separate dashboards for Patient, Doctor and Admin; a PHP REST API that owns authentication, approval gating, OTP verification and upload handling; and a MariaDB database hosted on an Apache/XAMPP server.

1.2 Problem

Today a discharged tracheostomy patient has no reliable channel to report daily status. Instructions go home on paper, diaries are kept inconsistently, and anything urgent travels as ad-hoc photos and calls over personal messaging apps. The consequences are predictable: readings are missing or incomparable, the doctor cannot tell whether care steps were performed, and early warning signs - a rising respiratory rate, falling SpO2, tube blockage - reach the clinician only when the patient calls, or arrives in emergency.

On the clinic side there is no organised view of out-of-hospital tracheostomy patients, no controlled way to onboard additional doctors, and no trusted place to publish the care-training material patients should follow. Existing generic apps - medication reminders, broad symptom diaries - cover none of this.

1.3 Aim and Objectives

The aim of TrachCare is to replace informal tracheostomy home-care tracking with a structured, monitored, same-day reporting loop. The objectives are:

- Give patients a simple guided daily report for tracheostomy care and vitals, replacing the paper diary.
- Give doctors a monitoring console - patient list, per-patient history and flagged abnormal reports.
- Give admins an approval workflow for new doctor accounts and a publishing tool for therapy videos.
- Keep authentication, uploads and data access safe enough for clinical use: prepared statements, no password leakage, approval gating.
- Ship a production-signed Android release suitable for Play Store distribution at the required API level.

1.4 Scope and Outcomes

The delivered product comprises the Flutter Android application, the PHP REST backend with its MariaDB schema on an Apache/XAMPP host, and the admin-curated video education library. Out of scope for this release are an iOS build, in-app chat or tele-consultation, and pharmacy or payment features - the product deliberately focuses on the daily reporting and monitoring loop.

Version 2.0.2 (build 10) is packaged as a signed App Bundle targeting API level 36 for Google Play, alongside a 69 MB universal APK for direct distribution.

1.5 Report Organisation

Chapter 2 analyses the gap the product fills and compares it to existing alternatives. Chapter 3 lists

requirements, Chapter 4 the technology stack, Chapter 5 the UX design process, Chapter 6 the architecture and database design, Chapter 7 the implementation, Chapter 8 project management, Chapter 9 testing, Chapter 10 deployment, Chapter 11 security and clinical responsibility, and Chapter 12 limitations and future work.

Chapter 2 - Gap Analysis, Product Description, Comparison and Uniqueness

2.1 Gap Analysis

2.1.1 How tracheostomy home care is monitored today

A patient discharged with a tracheostomy is typically given verbal and printed care instructions and asked to maintain a diary or return for scheduled reviews. Between visits, communication with the treating team happens over phone calls and personal messaging apps - a photo of the stoma, a voice message about a fever. The care team cannot see trends, cannot verify that care steps were performed, and has no structured record of what happened between two appointments.

Clinics that care for many such patients have no dedicated tool either: rosters are informal, junior doctors onboard ad-hoc, and educational material is shared as printed sheets or video links that get lost in chat histories.

2.1.2 Weaknesses of the existing method

Table 2.1: Weaknesses of paper-based tracheostomy care tracking

Weakness	Consequence
Paper diary kept by the patient	Entries are skipped, illegible or recorded only the night before a visit.
Photos shared over messaging apps	No structure, no history, mixed with personal chats, no consent trail.
Review only at scheduled visits	Deterioration between visits is invisible to the doctor.
No standard set of readings	Each patient reports different things; trending over time is impossible.
No alert for abnormal values	A dangerous reading sits unread until the next appointment.
Patients unsure of correct care	Verbal discharge instructions are forgotten within days.

2.2 Product Description

2.2.1 Users and roles

Table 2.2: Users and roles

Role	What they do in TrachCare
Patient	Signs in, submits the daily update (vitals, tube and surgery status, lab values), watches therapy videos, manages reminders and profile.
Doctor	Signs in after admin approval, adds patients with their baseline chart, reviews the patient list and daily reports, and sees flagged reports highlighted.
Admin	Approves or rejects doctor and admin accounts, manages doctor and patient lists, publishes therapy videos with thumbnails, and monitors the system.

2.2.2 Core flows

All three roles share one backend and one database. The login screen accepts either the registered email or the username. New self-registered doctors and admins stay PENDING until an existing admin approves them, and the login endpoints enforce that status - a rejected or unapproved account gets a clear explanation instead of a silent failure.

The daily report is the heart of the product: the patient answers a structured set of vitals and care fields, the entry lands in the doctor's queue the same day, and abnormal submissions are surfaced as flagged reports rather than buried in the list.

2.3 Comparison and Uniqueness

2.3.1 Existing alternatives

Table 2.3: Comparison of TrachCare with existing alternatives

Product	What it does	Gap for tracheostomy care
MyTherapy	Medication reminders and a general health diary.	No procedure-specific tracking, no doctor review loop.
Medisafe	Pill reminders with caregiver alerts.	Medication-focused; no vitals diary or clinician monitoring.
CareClinic	Broad symptom and measurement diary.	Generic tracking; nothing specific to tracheostomy care.
Patient.info	Health education articles.	Content only - no reporting, no connection to the care team.
WhatsApp calls/photos	Ad-hoc sharing between patient and doctor.	No structure, no history, no consent, no flagged alerts.

2.3.2 Uniqueness

- Tracheostomy-specific daily report: post-op day, tube name and size, surgery status, indication, comorbidities and lab values alongside vitals.
- A closed monitoring loop: every submission is reviewable by the doctor, with abnormal reports flagged rather than buried in a list.
- Guided education: admins publish care videos (tie change, dressing, suctioning) that patients stream in-app.
- Controlled onboarding: self-registration is allowed but privileged accounts activate only after admin approval.

2.4 Assumptions

- Patients or caregivers can operate a basic Android phone.
- The treating clinic runs or can deploy the bundled XAMPP backend.
- Connectivity is available at least once a day to submit the report.

Chapter 3 - Requirements

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Priority
FR-1	Role-based login for Patient, Doctor and Admin with username or email.	High
FR-2	Self-serve signup for all roles with OTP email verification.	High
FR-3	Password reset via OTP code and reset token.	High
FR-4	Admin approval/rejection of pending doctor and admin accounts.	High
FR-5	Patient submits a daily update: vitals, tube and surgery status, labs.	High
FR-6	Doctor adds a patient with baseline chart and photo.	High
FR-7	Doctor views patient list and per-patient daily reports.	High
FR-8	Flagged abnormal reports surface to the doctor.	High
FR-9	Admin adds a doctor account with photo.	Medium
FR-10	Admin publishes therapy videos with thumbnails; patients stream them.	Medium
FR-11	Users edit their profile details and photo.	Medium
FR-12	Doctor views flagged reports of his patients.	Medium
FR-13	Reminders and calendar views for patient care tasks.	Low

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

Attribute	Requirement	Priority
Security	Prepared statements on all queries; passwords not returned by any list endpoint; privileged roles gated on approval.	Mandatory
Performance	Lists and dashboards render within ~2 s on a mid-range phone over mobile data; upload size capped at 250 MB.	Mandatory
Reliability	Every form confirms only after a real server success; duplicate submits blocked; requests time out at 30 s.	Mandatory
Usability	Single-thumb forms, skeleton loaders, animated feedback, reduced-motion support for accessibility.	Mandatory
Compatibility	Android 7.0+ (minSdk 24), targetSdk 36; all ARM devices.	Mandatory
Maintainability	Shared UI kit and motion library; single API base URL configurable at build time.	Desirable

3.3 Constraints

- The production API is plain HTTP - the HTTPS virtual host on the server points at a different document root, so TLS is a server-side follow-up.
- Existing database rows predate the approval workflow - they default to APPROVED so no current user is locked out.

• Video assets are stored as files on the server with only paths in the database, matching the existing deployment layout.

Chapter 4 - Technology Stack

Table 4.1: Technology stack

Layer	Technology
Mobile app	Flutter 3.24 / Dart - Android
Backend API	PHP 8 REST endpoints returning JSON
Database	MariaDB / MySQL "trachcare" on XAMPP
Web server	Apache (XAMPP) - http://180.235.121.245/Trachcare
DB admin	phpMyAdmin
Video playback	media_kit (native players per ABI)
Media upload	image_picker / file_picker, multipart POST
Local storage	shared_preferences (session, remembered login)
Notifications	flutter_local_notifications + timezone
UI	google_fonts, sizer, toastification, shared motion library
Build & signing	Gradle, upload keystore (JKS), APK/AAB via flutter build
Distribution	Google Play Console - signed AAB, API 36

4.1 Frontend

The app is a single Flutter codebase. Shared components live in lib/components: a design-system UI kit (cards, fields, skeletons, empty states, roster screens, navigation) and a native-animation motion library providing the ECG trace, staggered reveals, shimmer loaders, async buttons with deduplication, success-check feedback and reduced-motion support. API calls sit in lib/Api with a single base URL overridable at build time via --dart-define.

4.2 Backend

The backend is PHP on Apache. A shared db.php centralises the connection and helpers: JSON input decoding, password verification that accepts both the legacy plaintext rows and password_hash values, approval-column management (added idempotently on first use) and the OTP table (created on first request). All queries use prepared statements, and PHP warnings are suppressed from output so they cannot corrupt JSON responses.

4.3 Database

MariaDB holds the trachcare schema: the patient chart (addpatients), doctor and admin accounts (doctorprofile, adminlogin), daily reports, the video library, spigotting status and the OTP lifecycle table. phpMyAdmin is used for administration; the report deployment package includes the exact table usage described in Chapter 6.

4.4 Tooling

Table 4.2: Hardware and software requirements

Item	Requirement
Patient/doctor device	Android phone, 2 GB RAM+, Android 7.0+
Server	XAMPP (Apache + MariaDB) host reachable over HTTP
Development	Windows PC, Flutter SDK 3.24, Android SDK 36, JDK 21
Signing	upload-keystoresiavnesan.jks + android/key.properties

Chapter 5 - User Experience Design

5.1 Process

The design follows a medical-professional visual language: a teal brand gradient, card-based surfaces, generous spacing and clear role-coloured accents. User research covered how discharged tracheostomy patients actually record care (paper diaries, phone photos) and how doctors triage out-of-hospital patients. The primary persona is an elderly post-surgical patient with a family caregiver; the secondary is the treating doctor reviewing several patient panels.

5.2 Wireframes

Paper wireframes defined the three role flows - patient daily check-in, doctor monitoring and admin approvals - before any high-fidelity work. The four primary flows are shown below; each was iterated once after feedback before implementation.

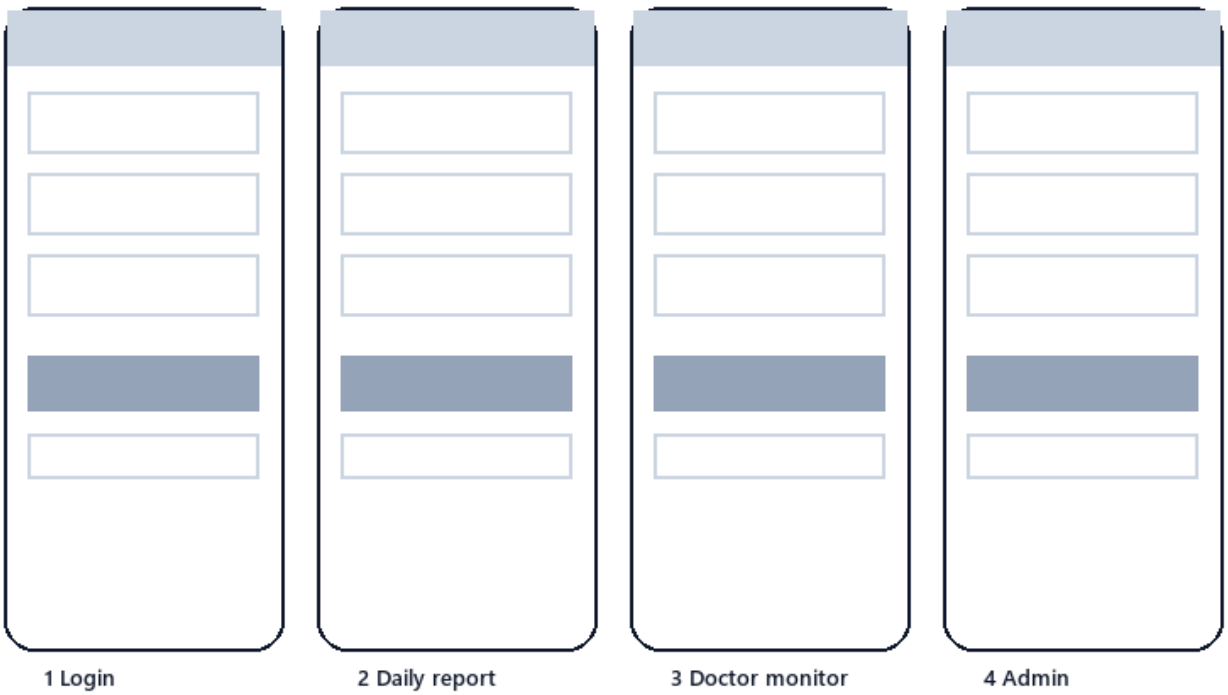


Figure 5.1: Early wireframes for login, daily report, doctor monitor and admin

5.3 User Flow

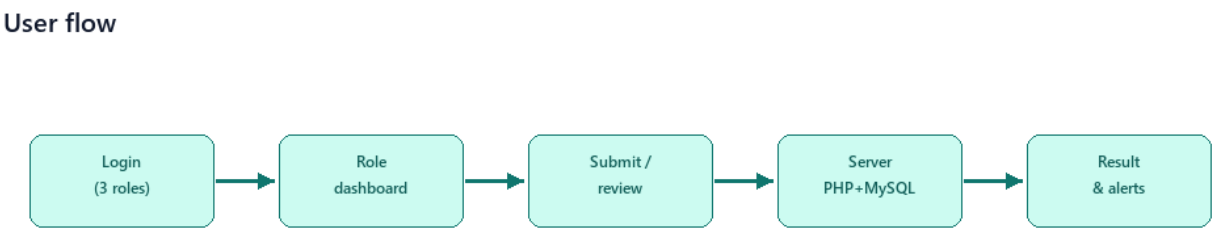


Figure 5.2: Role login leads to role dashboards, submissions reach the server and results come back as confirmations or alerts

5.4 Interaction States

Every asynchronous action has explicit states. Buttons show a spinner and de-duplicate taps while a request is in flight; success only displays after the server confirms; failed attempts leave the entered data in place. Lists render shimmer skeletons while loading, and empty screens float an illustration with a call to action. The system reduced-motion accessibility flag disables decorative animation for users who request it.

5.5 Design Review Findings

Table 5.1: Design review findings and changes

Finding	Change made
Profile avatar clipped under the app bar	Header rebuilt as a Stack with unclipped overflow; the full circle now shows.
Buttons allowed duplicate submissions	TcAsyncButton spinner + tap de-duplication across all forms.
Static grey placeholders while loading	Shimmer skeletons and staggered row reveals on every list.
Flat, motionless dashboards	ECG trace strip, pulse adherence ring and cascading sections added.
No animation on empty screens	Floating empty-state illustrations introduced.
Success shown before the server answered	All forms now confirm only on a real Status:true response.

Chapter 6 - System Architecture and Design

6.1 Architecture

TrachCare follows a three-tier client-server design. The Flutter app is the presentation tier; PHP endpoints on Apache form the application tier and own all business rules (authentication, approval gating, OTP lifecycle, upload handling); MariaDB is the data tier. The app never touches the database directly - every read and write goes through a JSON-speaking endpoint.

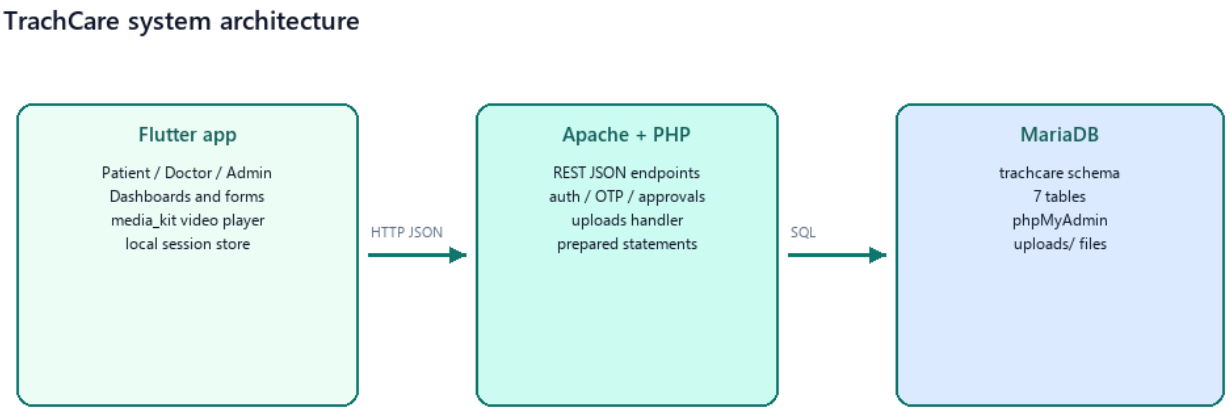


Figure 6.1: System architecture: Flutter app to Apache/PHP endpoints to MariaDB

Images and videos travel as multipart uploads; the server stores files under uploads/ and keeps only their paths in the database, which the app resolves back to absolute URLs when rendering. The API base URL is a single constant compiled into the app, overridable at build time.

6.2 Database Design



Figure 6.2: Database tables: patient chart, doctor/admin accounts, daily reports, video library, spigotting status and OTP codes

Table 6.1: Database tables and key fields

Table	Purpose / key fields
addpatients	Patient chart: patient_id, name, email, phone, password, age, gender, appointment, diagnosis, surgery status, post-op day, tube name/size, baseline vitals, labs (Hb, Na, K, Ca, bicarbonate, PT, aPTT, INR, platelets, LFT, RFT), image path, doctor_id.
doctorprofile	Doctor account: doctor_id, username, doctor_reg_no, email, phone_number, password, image_path, status, rejection_reason.
adminlogin	Admin account: id/doctor_id, username, email, password, status, rejection_reason.
dailyupdates	Daily report rows keyed by patient_id with the vitals and care fields submitted from the app.
patientvideotable	Therapy library: s.no, title, description, Video_url, Thumbnail_url.
spiotting_status	Spigotting/weaning status per patient.
otp_codes	OTP lifecycle: email, role, purpose, code, reset_token, expires_at, used (created on first request).

6.3 Data Flow

- Login: the app posts JSON {username, password} to the role endpoint; the server verifies credentials and approval status, returning Status + userInfo.
- Daily update: the patient form posts vitals to adddailyupdates.php; the doctor reads them through Dailyupdates.php / flaggedreports.php.
- Onboarding: signup endpoints create PENDING accounts after OTP verification; admin approve/reject endpoints flip the status column.
- Uploads: multipart images go to uploads/doctorimages or patient images; videos and thumbnails go to uploads/videos and uploads/thumbnails.

Chapter 7 - Implementation

7.1 API Endpoints

Table 7.1: PHP REST API endpoints

Endpoint	Method	Purpose
auth/patientAuth/PatientLogin.php	POST	Patient login (username or email).
auth/DoctorAuth/DoctorLogin.php	POST	Doctor login, enforces approval status.
auth/AdminAuth/AdminLogin.php	POST	Admin login, enforces approval status.
auth/{role}/*Signup.php	POST	Signup per role after OTP.
auth/request_otp.php	POST	Issue a 6-digit OTP (signup/reset).
auth/verify_otp.php	POST	Verify OTP, return reset token.
auth/reset_password_with_otp.php	POST	Set new password with token.
Admin/pending_{doctors,admins}.php	GET	List pending accounts.
Admin/{approve,reject}_{doctor,admin}.php	POST	Decide an account.
Admin/Doctorlist.php, patientlist.php	GET	Roster lists (passwords omitted).
Doctor/Patientslist.php	POST	Patients under a doctor.
Doctor/Addpatients.php	POST	Create patient chart (multipart).
Admin/Adddoctordetails.php	POST	Create doctor (multipart, validated).
Patient/adddailyupdates.php	POST	Submit the daily report.
Doctor/Dailyupdates.php	POST	Read patient reports.
Doctor/flaggedreports.php	POST	Abnormal flagged reports.
Admin/addvideos.php	GET/POST	Publish + list therapy videos.
Patient/editpatientprofile.php	POST	Update profile + photo.
Doctor/status.php, updatestatus.php	POST	Care/spigotting status.

7.2 Modules

Table 7.2: Module summary

Module	What it contains
Auth	Role login screens, signup wizard with OTP, forgot-password flow, session persistence.
Patient	Dashboard with adherence ring and ECG strip, daily update form, video library, reminders, profile.
Doctor	Dashboard with vitals strip, patient roster, add-patient chart, per-patient reports, flagged reports.
Admin	Console, doctor/patient management, add doctor, publish videos, approvals queue.
Shared	Design-system UI kit, motion library, API layer, datastore, reminder service.

7.3 Screens

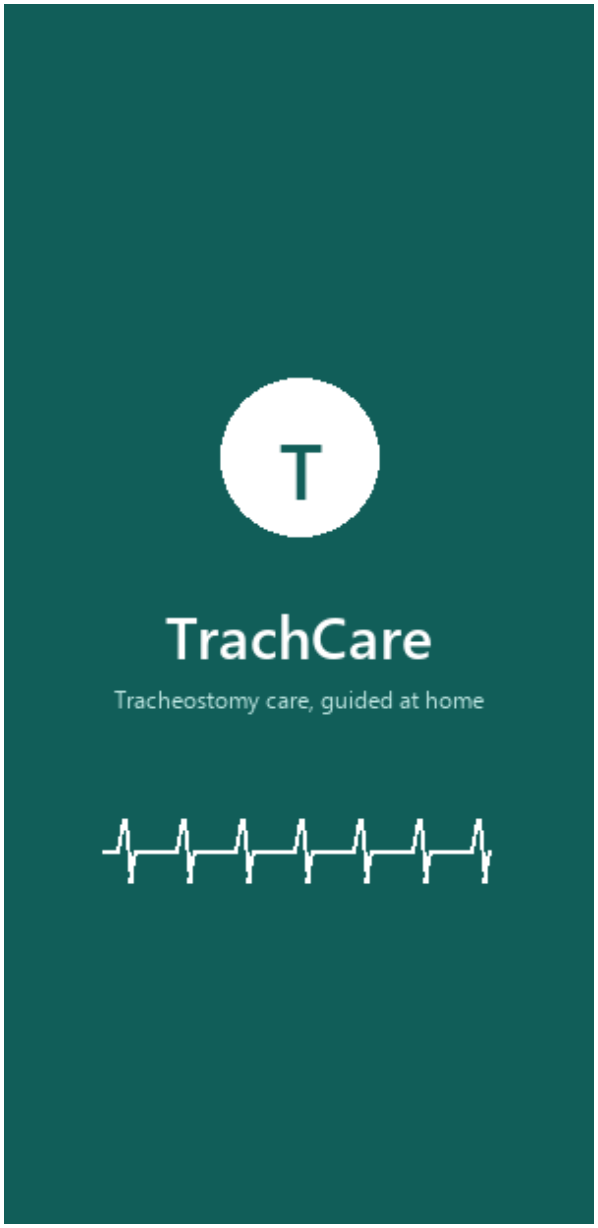


Figure 7.1: Splash screen: brand mark, tagline and an animated ECG trace

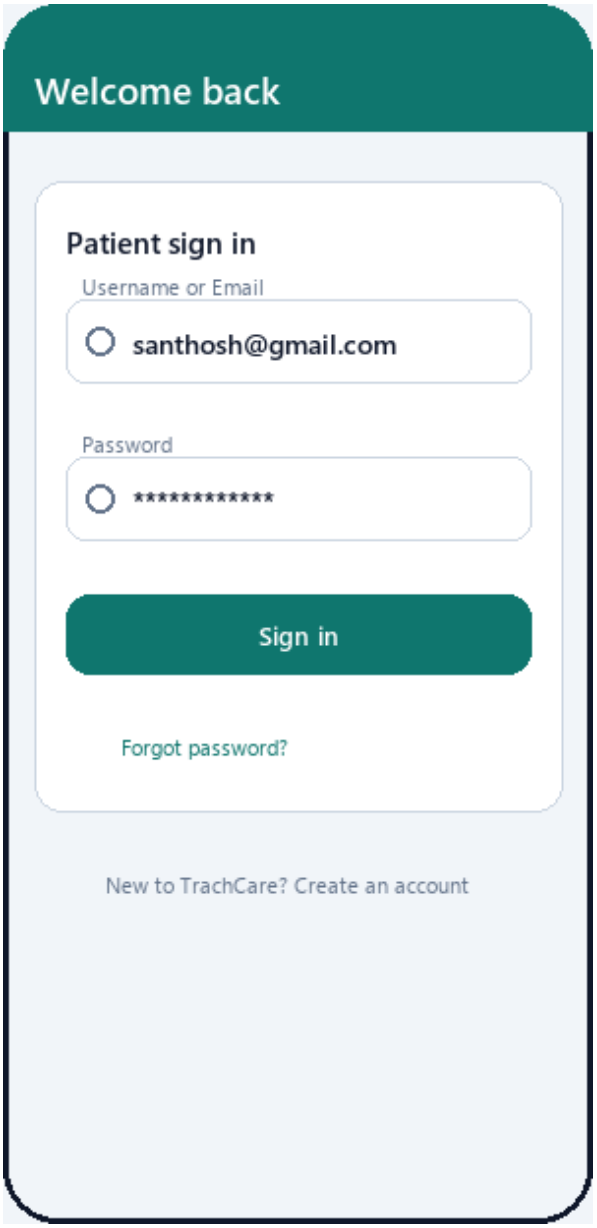


Figure 7.2: Patient sign-in: username or email + password, forgot-password link, signup entry

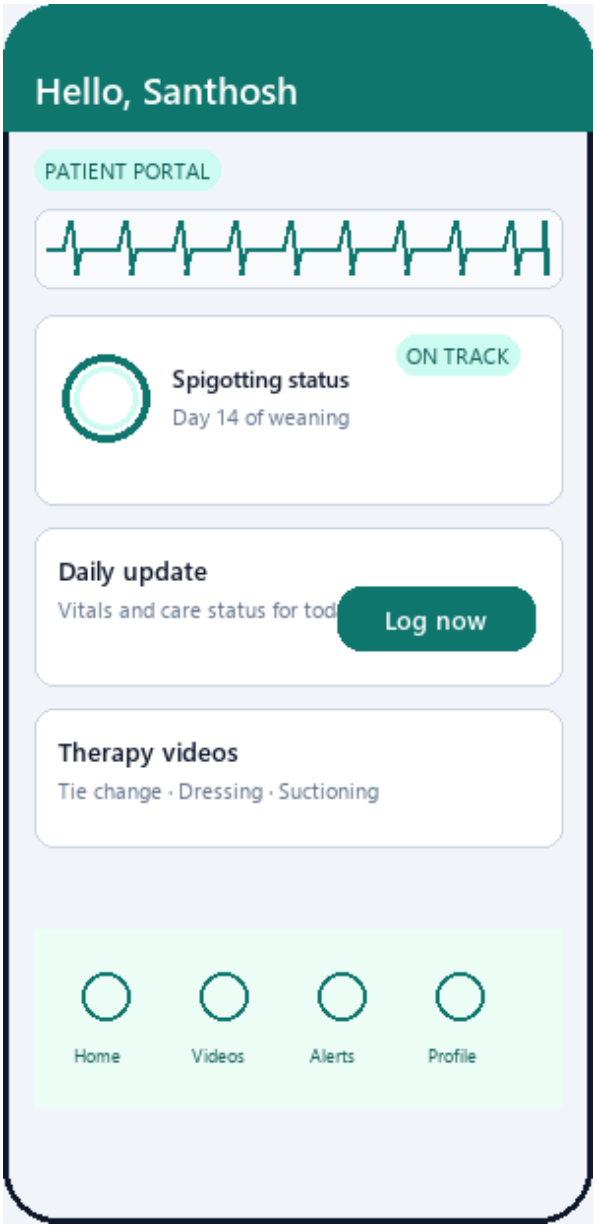


Figure 7.3: Patient dashboard: ECG strip, spigotting card, daily-update entry and video library

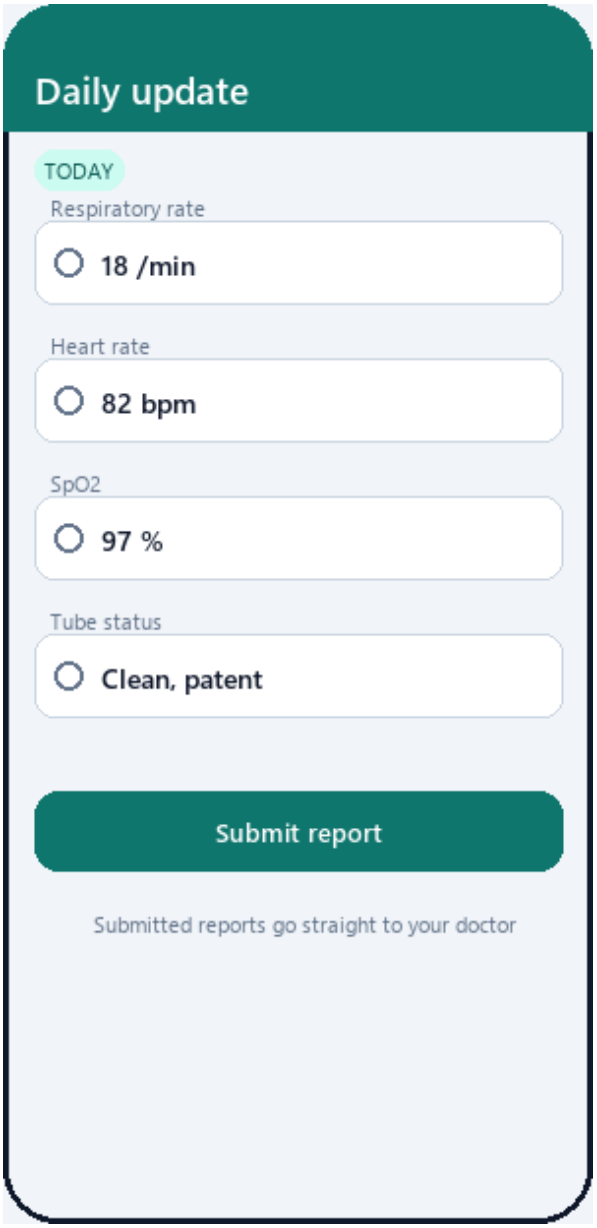


Figure 7.4: Daily update: structured vitals and tube-status fields submitted to the doctor

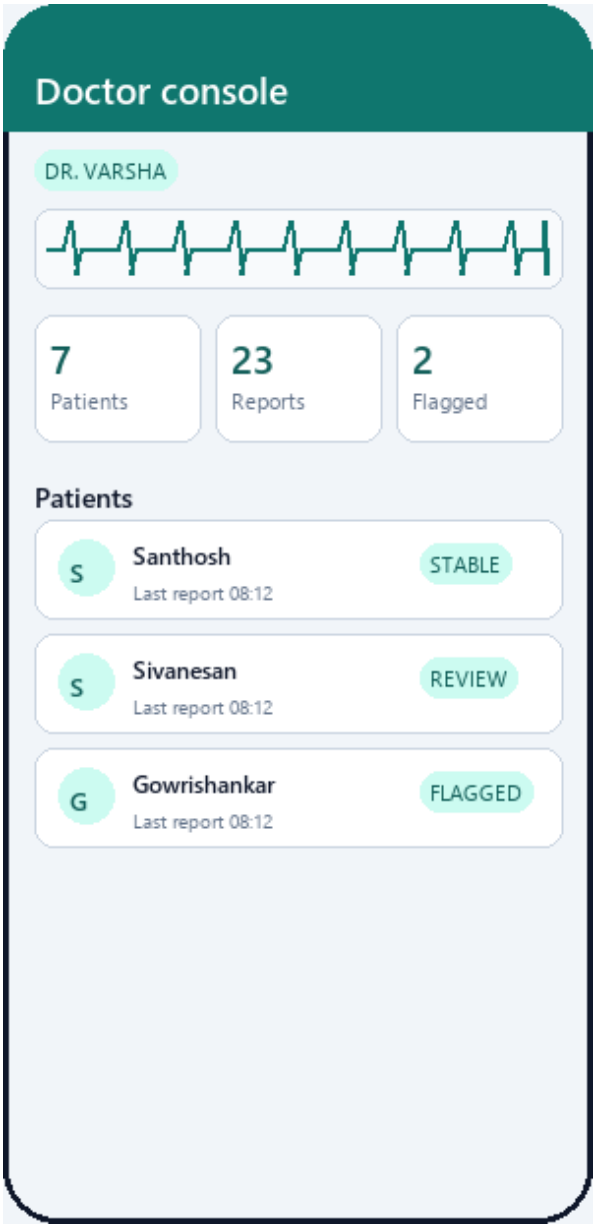


Figure 7.5: Doctor dashboard: vitals strip, counters and the monitored patient list with flagged entries

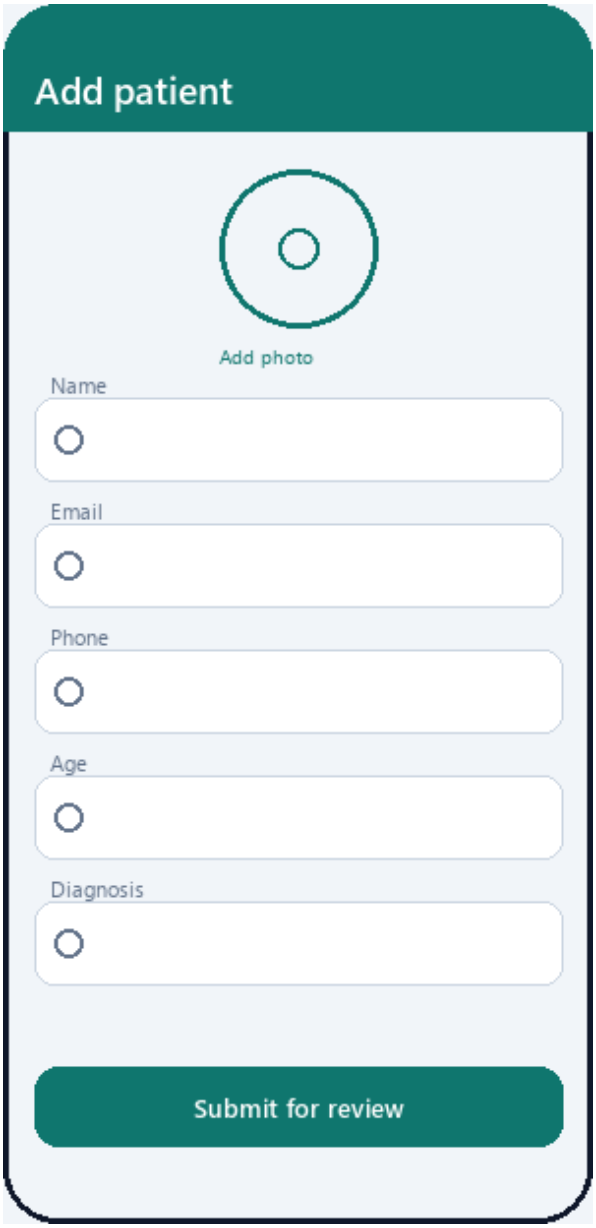


Figure 7.6: Add patient: photo capture and baseline chart fields with validation

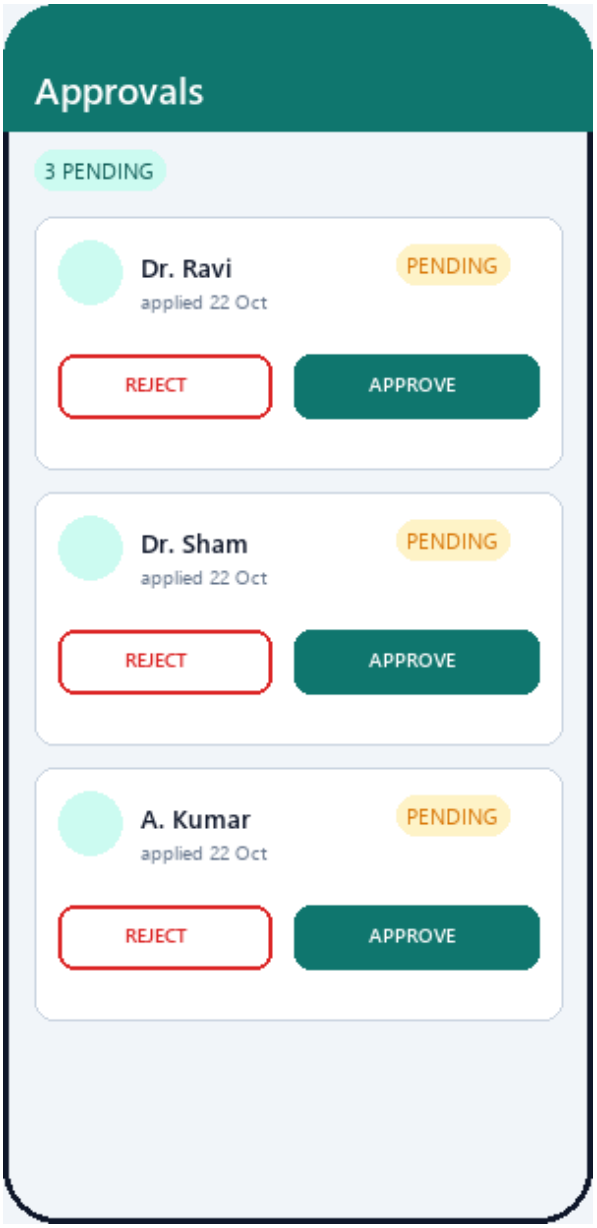


Figure 7.7: Admin approvals: pending doctor/admin accounts with per-card approve and reject actions

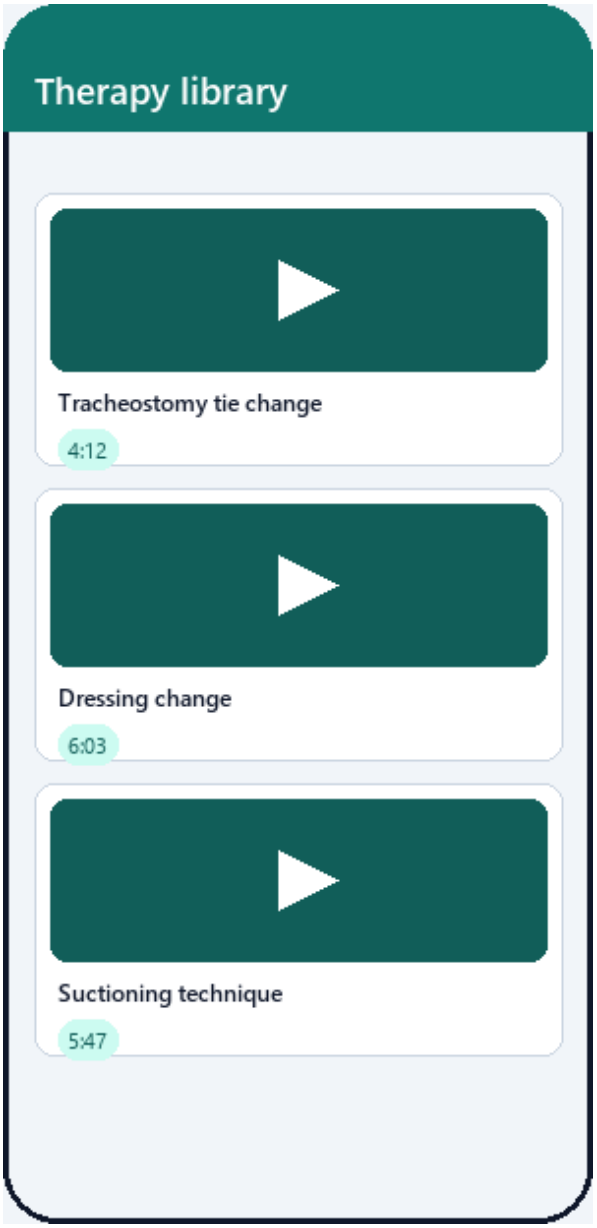


Figure 7.8: Therapy library: admin-published care videos streamed in-app

7.4 Notable Implementation Details

- Every submission function returns a real success flag parsed from the server response - forms only show confirmation after Status:true.
- All mutating API calls use prepared statements; list endpoints omit password fields entirely.
- Doctor and admin logins reject PENDING and REJECTED accounts with a clear message.
- Uploads are validated client-side (250 MB cap) and server-side.
- Animations use a shared native-Flutter motion library - no external animation assets, honouring reduced-motion.
- A 30-second timeout on every request turns dead connections into readable errors instead of indefinite spinners.

Chapter 8 - Project Management and Teamwork

8.1 Stages

Table 8.1: Project stages

Stage	Work
1. Analysis	Studied the discharge care workflow, existing app behaviour and live API responses; defined the three roles.
2. Design	Wireframes, design system, component library, screen contracts.
3. Backend	Reconstructed and hardened the PHP API: logins, signups, OTP, approvals, lists, uploads.
4. App build	Screens per role, shared UI kit, motion library, API integration.
5. Hardening	Fixed false-success forms, deduped submits, timeouts, approval gating, removed password leaks.
6. Release	Analyze-clean build, signed AAB/APK, API 36, Play Console packaging.

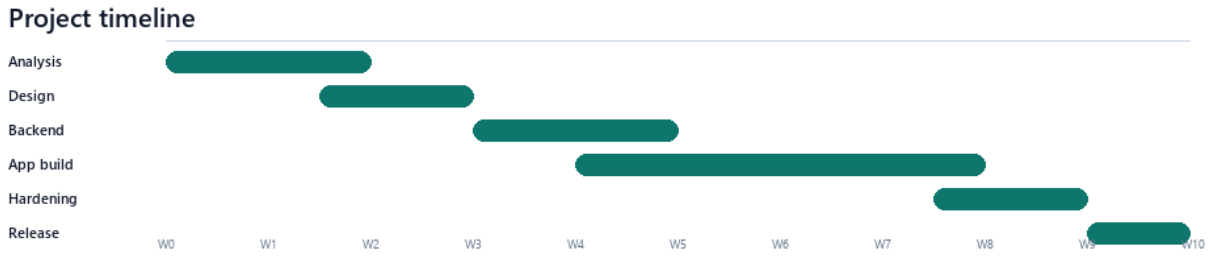


Figure 8.1: Working timeline across the project

8.2 Team and Tools

Table 8.2: Team roles and responsibilities

Area	Responsibility
App development	Flutter screens, components, animations, API layer.
Backend	PHP endpoints, database schema, upload handling, security hardening.
Design	Design system, wireframes, motion language.
Testing & release	Analyzer, PHP lint, endpoint testing, signed builds, Play Console.

Work was tracked with a todo list per phase and versioned in Git. Verification ran continuously: flutter analyze after each app change, php -l on every endpoint, and live API probes after each backend change - regressions were caught at the stage that introduced them rather than at release time.

Chapter 9 - Testing and Validation

9.1 What Was Tested

Table 9.1: Test cases and results

Area	Test	Result
Static analysis	flutter analyze on the full app	0 errors, 0 warnings
PHP syntax	php -l on all 22 backend files	All pass
Admin login	admin / valid password	Status:true, userInfo returned
Doctor login	registered email + password	Status:true
Doctor login	username instead of email	Status:true (fixed endpoint)
Patient login	registered email + password	Status:true
Approval gate	login as PENDING doctor	Blocked with message
List endpoints	Doctorlist / patientlist / Patientslist	Valid JSON, no password fields
Add patient form	submit valid chart	Success only on Status:true
Daily update	submit with server offline	Error shown, form data preserved
Profile save	patient/doctor/admin edit	Success dialog only on real save
Upload cap	pick file >250 MB	Rejected with size message
Release build	flutter build apk/appbundle	69 MB APK, 59.9 MB AAB, signed

9.2 Verification Approach

API behaviour was verified by direct requests to the live server before and after each backend change, checking exact JSON shape and status flags. The app was statically analysed after every change set, and the release bundle was verified with apksigner/jarsigner for the correct upload certificate, versionCode and targetSdkVersion 36.

A notable catch during testing: the live Adddoctordetails.php accepted empty requests and had silently written three blank doctor records into the production table - the replacement endpoint now validates every field, and the junk rows were identified for cleanup. Similarly, several forms were found to show success unconditionally; all now wait for the server's real status.

Chapter 10 - Deployment

10.1 Release Packaging

The app ships as a signed Android App Bundle (59.9 MB) for Play Console and a universal signed APK (69.2 MB) for direct distribution. Per-architecture splits are also produced: arm64-v8a (40 MB), armeabi-v7a (37 MB) and x86_64 (45 MB).

- Version: 2.0.2 (versionCode 10), applicationId com.simats.trachcare.
- targetSdkVersion 36, minSdkVersion 24.
- Signed with the existing upload keystore; fingerprint verified to match the Play Console requirement (SHA-1 A3:CB:EA:...:C1:E8).
- x86_64 native libraries excluded from release packaging (emulators only), keeping the universal APK small while covering every real ARM device.

10.2 Backend Deployment

The hardened backend ships as Trachcare_backend_fix.zip - 22 PHP files extracted into C:\xampp\htdocs\Trachcare\ alongside the existing folder structure. Deployment steps: back up the replaced files, update db.php credentials, upload, then verify logins for all three roles and the approval flow. An .htaccess raises PHP upload limits so therapy videos within the 250 MB cap are accepted.

10.3 Play Console

The AAB was produced for Play upload with the corrected signing certificate and a fresh versionCode after an earlier upload had been rejected - that earlier bundle had been signed by a different key and reused versionCode 7. The final bundle carries the expected certificate fingerprint, versionCode 10 and targetSdkVersion 36, clearing both rejections.

Chapter 11 - Security, Privacy and Clinical Responsibility

11.1 Security Controls

Table 11.1: Security threats and controls

Threat	Control
SQL injection	All queries use prepared statements via shared db.php helpers.
Password leakage	List endpoints no longer return password fields (they previously did, unauthenticated).
Unauthorised privileged access	Doctor/admin login enforces approval status; self-signup creates PENDING accounts.
Credential guessing	Login accepts username or email but returns a generic invalid-credentials message.
Corrupt API responses	display_errors off; warnings logged, not printed into JSON.
Bad uploads	Client-side 250 MB cap + extension whitelist and server-side validation on image/video uploads.
Junk records	Required-field validation added to Adddoctordetails.php (blank submissions had created empty doctors).

11.2 Clinical Responsibility

The app displays only data the patient or doctor entered - no vitals, diagnoses or prescriptions are invented for demonstration. Illustrative animations (the ECG strip, the pulse ring) are decorative and never presented as live measurements. The daily report asks for real clinical values, and abnormal submissions are flagged for the doctor to interpret - the app records and routes; the clinician decides.

11.3 Privacy

Patient data stays on the clinic server; the app stores only session identifiers and an opt-in remembered login locally. Passwords in the database are currently plaintext - a legacy constraint kept for compatibility with the deployed data. The new login code already accepts password_hash values so a migration can roll out per-user without breaking anyone.

Chapter 12 - Limitations, Future Work and Conclusion

12.1 Limitations

- The API is plain HTTP - the HTTPS virtual host serves a different document root, so TLS termination still needs server-side setup.
- Existing database rows store plaintext passwords; hashing support is ready but the migration step is pending.
- Android only - no iOS build in this release.
- x86_64 release packages are excluded, so release builds do not run on Intel emulators/tablets.
- Chat and tele-consultation between patient and doctor are not yet implemented.

12.2 Future Work

Table 12.1: Future enhancements

Enhancement	Benefit
Migrate passwords to password_hash	Removes the last plaintext credential store.
HTTPS on the API host	Encrypts credentials and clinical data in transit.
Push notifications (FCM)	Alerts doctors to flagged reports and reminds patients to submit.
In-app chat / video consult	Direct doctor-patient contact for urgent review.
iOS release	Same Flutter codebase covers Apple devices.
Offline-first daily report	Queues submissions when connectivity drops.

12.3 Conclusion

TrachCare turns the paper-based tracheostomy diary into a monitored, structured daily workflow. Patients get a guided check-in and a video library; doctors get same-day visibility with flagged alerts; admins get controlled onboarding and publishing. The backend was reconstructed and hardened, every form now confirms real results, and the app ships as a signed, API-36 release ready for Google Play.

References

[1] Flutter documentation - https://docs.flutter.dev

[2] Dart language tour - https://dart.dev/guides

[3] PHP manual - https://www.php.net/docs.php

[4] MariaDB documentation - https://mariadb.com/kb/en/documentation/

[5] Android target API level requirements - https://developer.android.com/google/play/requirements/target-sdk

[6] image_picker plugin - https://pub.dev/packages/image_picker

[7] media_kit video playback - https://pub.dev/packages/media_kit

[8] Google Play Console help - https://support.google.com/googleplay/android-developer

[9] Tracheostomy care guidance - https://www.tracheostomy.org.uk

[10] OWASP cheat sheet: SQL injection prevention - https://cheatsheetseries.owasp.org